

CSCE 637 COMPLEXITY THEORY · MODULE 12 · LECTURE NOTES

Fine-Grained and Parameterised

The complexity theory that answers your actual question.

MODULE OUTCOMES

- Explain fixed-parameter tractability and give examples.
- Choose a parameter for a real problem.
- Explain kernelisation and its practical value.
- Explain fine-grained complexity and the standard hypotheses.
- Explain why this is the practically useful part of the field.

1 Parameterised complexity

FIXED-PARAMETER TRACTABLE: EXPONENTIAL IN A PARAMETER, POLYNOMIAL IN THE SIZE

A problem is fixed-parameter tractable with respect to a parameter k if it is solvable in time $f(k) \cdot n^c$ — any function of k , however horrible, times a polynomial in n whose exponent does not depend on k . The exponent being independent of k is the essential part; n^k is not FPT.

So vertex cover is NP-complete and fixed-parameter tractable: $O(2^k \cdot n)$ finds a cover of size k , or reports that none exists — branch on each edge, taking one endpoint or the other. Which is entirely practical for k up to about 40 on a graph of any size whatsoever.

And that is the right question for most real problems, because the thing that is large and the thing that is hard are usually different — a huge graph with a small solution, a long formula with few genuinely interacting variables, a big instance with low treewidth. Worst-case complexity conflates them by measuring only n .

Which makes parameterised complexity the practically useful branch of the field, and it is the one most often omitted from a complexity course — a situation this module exists to correct.

Parameter	What it makes tractable	Note
Solution size k	Vertex cover, feedback vertex set, k -path, hitting set.	$O(2^k n)$, and better for several — vertex cover is down to about 1.27^k . The classic parameter.
Treewidth	Almost every graph problem.	Courcelle's theorem: any property expressible in monadic second-order logic is decidable in linear time on graphs of bounded treewidth. See the note.

Parameter	What it makes tractable	Note
Number of variables	Integer programming.	Lenstra: integer programming is FPT in the dimension — which is why small-dimension integer programs are easy regardless of the coefficients.
Clause-to-variable ratio	SAT, empirically.	Not a theorem — but a real and sharp phase transition around 4.27 for 3-SAT, with instances far from it being easy in practice.
Vertex cover number	More than treewidth does.	A stronger parameter — bounded vertex cover number implies bounded treewidth — and correspondingly rarer to find small.
Distance from triviality	The general design pattern rather than a specific parameter.	How many edges from being planar, how many vertices from being bipartite, how many clauses from being 2-SAT. The most transferable idea in the table.

Treewidth and Courcelle's theorem are the most powerful combination here — any property expressible in monadic second-order logic is linear-time decidable on graphs of bounded treewidth, which covers an enormous range of problems in one result and requires no per-problem algorithm design. **The constants are terrible and the existence result is frequently enough to justify building the dynamic program by hand,** which is how it is used in practice.

2 Kernelisation

A KERNEL is a polynomial-time preprocessing that reduces any instance to an EQUIVALENT instance whose size is bounded by a function of k alone -- independent of n .

VERTEX COVER, a size- k^2 kernel:

- any vertex of degree $> k$ MUST be in the cover
(otherwise all its neighbours must be, which exceeds k)
-> take it, decrement k , remove it
- any isolated vertex is irrelevant -> delete it
- after these rules, if more than k^2 edges remain there
is no cover of size k -- each of k vertices now covers at most k edges
-> so the surviving kernel has at most k^2 edges

THEN SOLVE THE KERNEL by brute force. Total time is polynomial in n , plus $f(k)$ on a tiny instance.

WHY THIS MATTERS PRACTICALLY: kernelisation is PREPROCESSING WITH A PROOF. Every real solver does preprocessing (CSCE 669 Module 09 section 4); a kernel is preprocessing with a stated guarantee about how much it reduces and why the answer is preserved.

AND A PROBLEM IS FPT IF AND ONLY IF IT HAS A KERNEL, which is a clean characterisation and means that looking for one is never wasted effort.

3 Fine-grained complexity

FINE-GRAINED COMPLEXITY ASKS FOR THE EXPONENT, AND PROVES LOWER BOUNDS CONDITIONALLY

'Polynomial' is not the question when n is a billion. n^2 and n^3 are both polynomial and only one of them is feasible — and a great deal of real engineering lives in exactly that gap (Module 01 §4).

So the field assumes a specific hardness hypothesis and derives exact exponents from it. SETH: satisfiability of k -SAT requires essentially 2^n time as k grows. 3SUM requires n^2 . All-pairs shortest paths requires n^3 .

And from those follow tight lower bounds for problems that are firmly in P: under SETH, edit distance requires $n^{2-o(1)}$ time — so the textbook dynamic programming algorithm is essentially optimal, and the decades spent looking for a subquadratic one were spent against a wall.

Which is exactly the practically useful kind of result: it tells you whether your quadratic algorithm can be improved — a question that 'it is in P' cannot even address, and one that an engineer with a slow program actually has.

Hypothesis	What it says	What it implies
SETH	k -SAT requires 2^n time as k grows.	Edit distance and longest common subsequence require n^2 ; orthogonal vectors is hard, and orthogonal vectors is the usual intermediate problem.
3SUM	Finding three numbers summing to zero requires n^2 .	A large family of computational geometry lower bounds (CSCE 620) — collinearity, polygon containment, visibility.
APSP	All-pairs shortest paths requires n^3 .	Negative triangle detection, graph radius, replacement paths, and second shortest path — all equivalent to APSP under subcubic reductions.
ETH	3-SAT requires $2^{\Omega(n)}$.	No subexponential algorithms for a wide range of NP-complete problems, and tight bounds on FPT running times.

These are conjectures and the results derived from them are conditional — but they are the only available route to statements like 'your quadratic algorithm is optimal', which unconditional theory cannot touch at all (Module 09 §2's $5n$ bound is the measure of how little is unconditional). So the conditionality is real and the alternative is no answer whatsoever, which makes the trade worth taking.

4 Why this is the part that pays

- 1 · Find the parameter. Solution size, treewidth, dimension, distance from an easy case (§1's table) — and then check whether it is actually small in your instances, which is a measurement rather than a theorem.
- 2 · Look for a kernel, or use a solver's presolve and measure how much it reduces (§2). The reduction is frequently dramatic and is almost always worth measuring.

- **3 · Check the approximation picture** (Module 11 §4) — both sides, which takes twenty minutes.
- **4 · Check the fine-grained lower bound** if your problem is polynomial and your implementation is too slow (§3) — it may tell you to stop optimising and change the problem.
- **5 · And only then consider the worst-case classification**, which is where most courses start. **The ordering is deliberate and is the opposite of the usual one — the parameterised question is answerable and actionable, and the worst-case one is frequently neither.**

WHY THIS IS THE PART OF COMPLEXITY THEORY THAT PAYS

Worst-case classification answers 'is there a polynomial-time algorithm correct on all inputs' — which is rarely the question you have, and is the question Modules 02 through 05 are about.

Parameterised complexity answers 'is there an algorithm fast on instances like mine', which usually is the question — and it answers it with a theorem rather than a benchmark.

And fine-grained complexity answers 'can my quadratic algorithm be improved', which is the question an engineer with a slow program actually asks and which no other branch addresses.

So the two newest branches are the two most useful — and they are the two most often omitted from a complexity course, which is a reasonable complaint about how the subject is taught and the reason this module is placed where it is rather than as an appendix.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Cygan et al. — Parameterized Algorithms (free PDF)	Free in full, and the reference for §1 and §2 — the most practically useful book in this course.
Williams — On Some Fine-Grained Questions in Algorithms and Complexity (free survey)	§3's hypotheses and their consequences , surveyed by one of the field's founders.
Bringmann — Fine-Grained Complexity Theory (free lecture notes)	§3 taught , with the edit-distance lower bound worked through.
Downey & Fellows — Fundamentals of Parameterized Complexity	The founders' treatment , including the W-hierarchy of parameterised intractability that this module does not cover. Library copy.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Implement the $O(2^k n)$ vertex cover algorithm** and run it on a large graph with a small cover.
- 2 **Implement the vertex cover kernel** and measure the reduction on real graphs.
- 3 **Verify the k^2 bound** empirically.
- 4 **Compute the treewidth** (or an upper bound) of three graphs from your own work.
- 5 **Write a dynamic program over a tree decomposition** for one problem.
- 6 **Identify a parameter for a hard problem you care about** and measure whether it is small in your instances.
- 7 **Implement edit distance** and confirm its quadratic scaling.
- 8 **Look up the SETH-based lower bound** and state what it means for your implementation.
- 9 **Find a 3SUM-hard geometry problem** and relate it to CSCE 620.
- 10 **Apply §4's five steps** to one problem and report which step was most useful.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Define fixed-parameter tractability precisely, including why the exponent must be independent of k .
- 2 Why is vertex cover both NP-complete and practical?
- 3 Why is the parameterised question the right one for real problems?
- 4 Name six parameters and what each unlocks.
- 5 State Courcelle's theorem and why it is so leveraged.
- 6 Define a kernel and give the vertex cover one.
- 7 What is the characterisation of FPT in terms of kernels?
- 8 What does fine-grained complexity ask, and name four hypotheses.
- 9 What does SETH imply about edit distance?
- 10 Give the five-step procedure and say why the ordering is reversed.